

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Лабораторная работа №2
«Тестирование ПО методом белого и чёрного ящика»
по дисциплине
«Методы тестирования программного обеспечения»

Студент,

группы 5130201/30101

_____ Мелешенко С. И.

Преподаватель

_____ Курочкин М. А.

« _____ » _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	4
1 Постановка задачи	5
2 Описание методов тестирования «черного» и «белого» ящиков	6
2.1 Метод черного ящика	6
2.1.1 Эквивалентное разбиение	6
2.1.2 Анализ граничных значений	7
2.1.3 Причинно-следственные диаграммы	7
2.2 Метод белого ящика	9
2.2.1 Покрытие операторов	9
2.2.2 Покрытие решений	10
2.2.3 Покрытие условий	10
2.2.4 Покрытие решений и условий	10
2.2.5 Комбинаторное покрытие условий	10
3 Тестирование программы №1	11
3.1 Описание программы	11
3.2 Тестирование методом «белого ящика»	13
3.2.1 Покрытие операторов	13
3.2.2 Покрытие решений	14
3.2.3 Покрытие условий	15
3.2.4 Покрытие решений и условий	16
3.2.5 Комбинаторное покрытие условий	17
3.2.6 Результаты тестирования методом «белого ящика»	17
4 Тестирование программы №2	19
4.1 Формальное описание программы	19
4.2 Тестирование методом «чёрного ящика»	21
4.2.1 Разбиение на классы эквивалентности	21
4.2.2 Анализ граничных значений	23
4.2.3 Причинно-следственная диаграмма	25
4.2.4 Результаты тестирования методом «чёрного ящика»	28
5 Заключение	29

Введение

Одним из ключевых этапов в современной разработке программного обеспечения является тестирование. Существует несколько видов тестирования: модульное, интеграционное, функциональное, системное и приемочное. В данной работе будет рассмотрено модульное тестирование.

Модульное или unit-тестирование является основным видом тестирования, с которого практически всегда начинается проверка корректности работы программы. Этот вид тестирования включает проверку отдельных блоков ПО: классов, модулей и функций. Цель модульного тестирования – выявить ошибки, то есть несоответствия поведения модуля его спецификации.

Существует два основных подхода к unit-тестированию: методы «чёрного» и «белого ящика».

В данной работе будут рассмотрены оба метода.

1 Постановка задачи

Цель работы: провести тестирование двух программ методами «черного ящика» и «белого ящика».

Задачи:

- 1) изучить методы модульного тестирования, в частности методы «белого ящика» и «черного ящика»;
- 2) разработать тесты для двух программ, и провести их тестирование;
- 3) проанализировать результаты тестирования.

2 Описание методов тестирования «черного» и «белого» ящиков

Для метода белого ящика необходим доступ к исходному коду и знание внутренней структуры программы, а для метода чёрного ящика – только спецификация (входные/выходные данные, требования).

2.1 Метод черного ящика

Одной из важнейших стратегий тестирования является тестирование методом черного ящика, которое также называют тестированием, управляемым данными, или тестированием, управляемым входом и выходом. В соответствии с этим методом программа рассматривается как «черный ящик», внутреннее поведение и структура которого не имеют никакого значения. Вместо этого все внимание фокусируется на выяснении обстоятельств, при которых поведение программы не соответствует спецификации.

Требуется протестировать программу с помощью разбиения множества входных данных на эквивалентные классы, анализа значений на их границах и построения причинно-следственных диаграмм.

Далее рассматриваются три приёма тестирования черного ящика.

2.1.1 Эквивалентное разбиение

Хорошо спроектированный тест должен покрывать значительную часть других возможных тестов, т.е. предоставлять некую информацию относительно наличия или отсутствия ошибок в ситуациях, не охватываемых данным конкретным набором входных значений. Поэтому следует попытаться разбить всю область входных данных программы на конечное число классов эквивалентности. Это позволит с достаточной долей уверенности полагать, что тестирование представительного значения данного класса эквивалентно тестированию любого другого значения, принадлежащего тому же классу.

Определение классов эквивалентности сводится к последовательному рассмотрению каждого из входных условий (обычно это предложение или фраза, приведенные в спецификации) и разбиению его на две или несколько групп. Определяется два типа классов эквивалентности: допустимые классы эквивалентности, представляющие допустимые входные данные программы и недопустимые классы эквивалентности, представляющие все остальные возможные состояния условий (т.е. недопустимые входные значения).

Определение тестов по построенным классам эквивалентности состоит из трех этапов:

- 1) назначить каждому классу эквивалентности уникальный номер;
- 2) записать новые тесты, охватывающие как можно большее количество оставшихся неохваченными допустимых классов эквивалентности, пока не будут покрыты все допустимые классы;

3) записать новые тесты, каждый из которых охватывает один и только один из оставшихся неохваченными недопустимых классов эквивалентности, пока не будут покрыты все недопустимые классы.

2.1.2 Анализ граничных значений

Граничные условия — это ситуации, возникающие в области граничных значений входных и выходных классов эквивалентности. Анализ граничных значений отличается от методики разбиения на классы эквивалентности в двух отношениях.

1. Вместо того чтобы выбирать любой элемент класса эквивалентности в качестве представителя всего класса, анализ граничных значений требует выбирать такой элемент или элементы, которые обеспечивают тестирование каждой границы класса.

2. При создании тестов внимание фокусируется не только на входных условиях (пространство входных значений), но и на пространстве результатов (выходные классы эквивалентности).

2.1.3 Причинно-следственные диаграммы

Причинно-следственная диаграмма представляет собой формальный графический язык, на который транслируется спецификация, написанная на естественном языке. Базовая нотация причинно-следственных диаграмм представлена на рисунке 1. Каждый узел диаграммы может находиться в двух состояниях: 0 или 1, где 0 представляет состояние «отсутствует», а 1 — «присутствует».

Метод позволяет решить проблему того, что метод анализа граничных значений и разбиения данных на классы эквивалентности не исследует комбинации входных условий.

Для построения тестов используется процесс, включающий несколько этапов.

1. Спецификация разбивается на части, с которыми легче работать.
2. В спецификации определяются причины и следствия.

Причина — это отдельное входное условие или класс эквивалентности входных условий.

Следствие — это выходное условие или преобразование системы (долговременное воздействие, которое входное условие оказывает на состояние программы или системы).

Причины и следствия определяются путем последовательного чтения спецификации и подчеркивания тех слов или фраз, которые описывают причины и следствия. Каждой причине и каждому последствию присваивается уникальный номер.

3. Семантическое содержание спецификации анализируется и преобразуется в булевый граф, связывающий причины и следствия. Полученный граф называется причинно-следственной диаграммой.

4. Диаграмма снабжается примечаниями, задающими ограничения и описывающими комбинации причин и (или) следствий, реализация которых невозможна из-за

синтаксических или внешних ограничений.

- Ограничение E требует, чтобы всегда выполнялось условие, в соответствии с которым только a или только b может быть равно 1 (a и b не могут быть равны 1 одновременно, но обе величины могут быть равны 0).

- Ограничение I требует, чтобы по крайней мере одна из величин, a , b или c , была равна 1 (a , b и c не могут быть равны 0 одновременно).

- Ограничение O требует, чтобы одна и только одна из величин, a или b , была равна 1.

- Ограничение R требует, чтобы a было равно 1, только если b равно 1 (т.е. a не может быть равно 1, если b равно 0). На рисунке 2 представлены символы ограничений.

5. Путем методичного прослеживания состояний условий диаграмма преобразуется в таблицу решений с ограниченными входами. Каждый столбец таблицы решений соответствует тесту.

6. Столбцы таблицы решений преобразуются в тесты.

Базовая нотация причинно-следственных диаграмм представлена на Рис. 1.

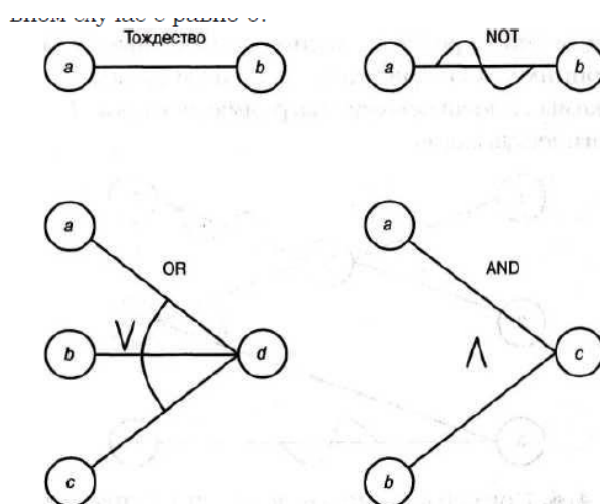


Рис. 1. Базовая нотация, используемая в причинно-следственных диаграммах.

Алгоритм заполнения очередного столбца таблицы решений приведен ниже.

1. Выбрать следствие, которое должно находиться в состоянии 1.

2. Продвигаясь вдоль линий диаграммы, ведущих к данному узлу, в обратном направлении, производится поиск комбинаций причин (подчиненных ограничениям), которые устанавливают данное следствие в 1.

- Если путь обратной трассировки проходит через узел типа OR, выход которого должен принимать значение 1, то не следует одновременно устанавливать в 1 более одного входа в этот узел.

- Если путь обратной трассировки проходит через узел типа AND, выход которого должен принимать значение 0, то, конечно же, должны быть перечислены все комбинации входов, приводящие к установке выхода в 0. Однако при исследо-

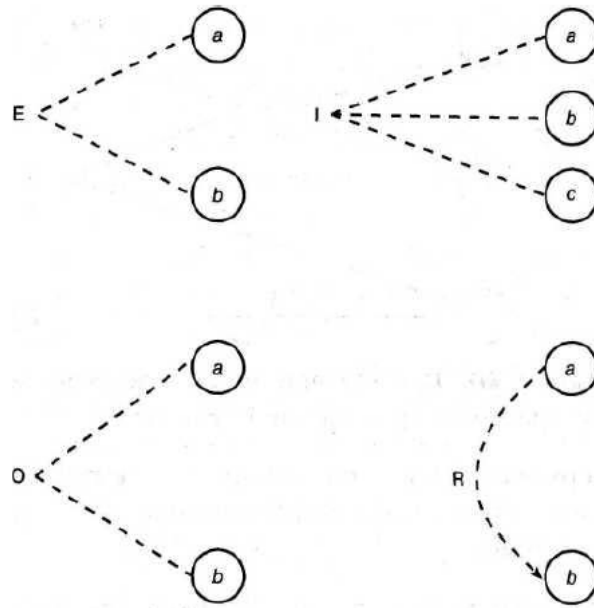


Рис. 2. Базовая нотация ограничений, используемая в причинно-следственных диаграммах.

вании ситуации, в которой один вход равен 0, а один или несколько других входов равны 1, перечислять все условия, при которых другие входы находятся в состоянии 1, не обязательно.

- Если путь обратной трассировки проходит через узел типа AND, выход которого должен принимать значение 0, то необходимо указать лишь одно условие, при котором все входы являются нулями.

3. Для каждой комбинации причин создается столбец в таблице решений.

4. Определить для каждой комбинации состояния всех других следствий и поместить их в соответствующий столбец таблицы.

2.2 Метод белого ящика

В стратегии тестирования методом белого ящика, называемой также тестированием, управляемым логикой программы, разрешается исследовать внутреннюю структуру программы. Исходя из этой стратегии, тестировщик подбирает тестовые данные путем анализа логики программы.

Требуется протестировать программу ниже приведенными методами.

2.2.1 Покрытие операторов

Критерием покрытия является выполнение каждого оператора программы хотя бы один раз. Это необходимое, но недостаточное условие для приемлемого тестирования по принципу белого ящика

2.2.2 Покрывтие решений

В соответствии с этим критерием количество тестов должно быть таким, чтобы каждое условие в программе хотя бы раз принимало как значение true (истина), так и false (ложь). Иными словами, каждая логическая ветвь каждой инструкции ветвления в программе должна быть выполнена хотя бы один раз.

2.2.3 Покрывтие условий

В соответствии с этим критерием количество тестов должно быть таким, чтобы каждое из элементарных условий, образующих проверочное выражение в точке ветвления, принимало каждое из двух возможных логических значений, true и false, по крайней мере один раз.

2.2.4 Покрывтие решений и условий

Согласно этому критерию набор тестов является достаточно полным, если удовлетворяются следующие требования: каждое условие в решении принимает каждое возможное значение по крайней мере один раз, каждый возможный исход решения проверяется по крайней мере один раз и каждой точке входа управление передается по крайней мере один раз.

2.2.5 Комбинаторное покрытие условий

Этот критерий требует создания такого количества тестов, при котором каждая возможная комбинация результатов вычисления условий в каждом решении и каждая точка входа проверяются по крайней мере один раз.

3 Тестирование программы №1

3.1 Описание программы

Автор: Чиботарь Андрей

Название: Бинарный поиск элемента в массиве

Дано:

- 1) n – размер массива;
- 2) A – упорядоченный по возрастанию массив целых чисел;
- 3) key – искомое число.

Требуется: найти и вывести индекс элемента key в массиве A или сообщить, что элемент отсутствует.

Ограничения:

- 1) n является целым числом в диапазоне $[1; 1000]$.
- 2) Элементы $A[i]$ и key – целые числа в диапазоне $[-2^{31}; 2^{31} - 1]$.
- 3) Массив A должен быть отсортирован по возрастанию.

Спецификация:

Входные данные			Ожидаемый результат	Комментарий
n	Массив A	key		
5	1 3 5 7 9	5	2	Элемент находится ровно в середине массива (индексация с 0)
5	1 3 5 7 9	1	0	Элемент находится в начале массива
5	1 3 5 7 9	9	4	Элемент находится в конце массива
5	1 3 5 7 9	4	-1 (Элемент не найден)	Искомое элемента нет в массиве
5	9 7 5 3 1	5	Ошибка / Непредсказуемо	Нарушено условие: массив не отсортирован по возрастанию
0	(пусто)	5	Ошибка размера	Размер массива должен быть в диапазоне $[1; 1000]$
1001	1 2 ... 1001	1000	Ошибка размера	Размер массива должен быть в диапазоне $[1; 1000]$

5	1 3 5 7 9	-2147483648	-1 (Элемент не найден)	Поиск минимально возможного целого числа (-2^{31})
---	-----------	-------------	------------------------	--

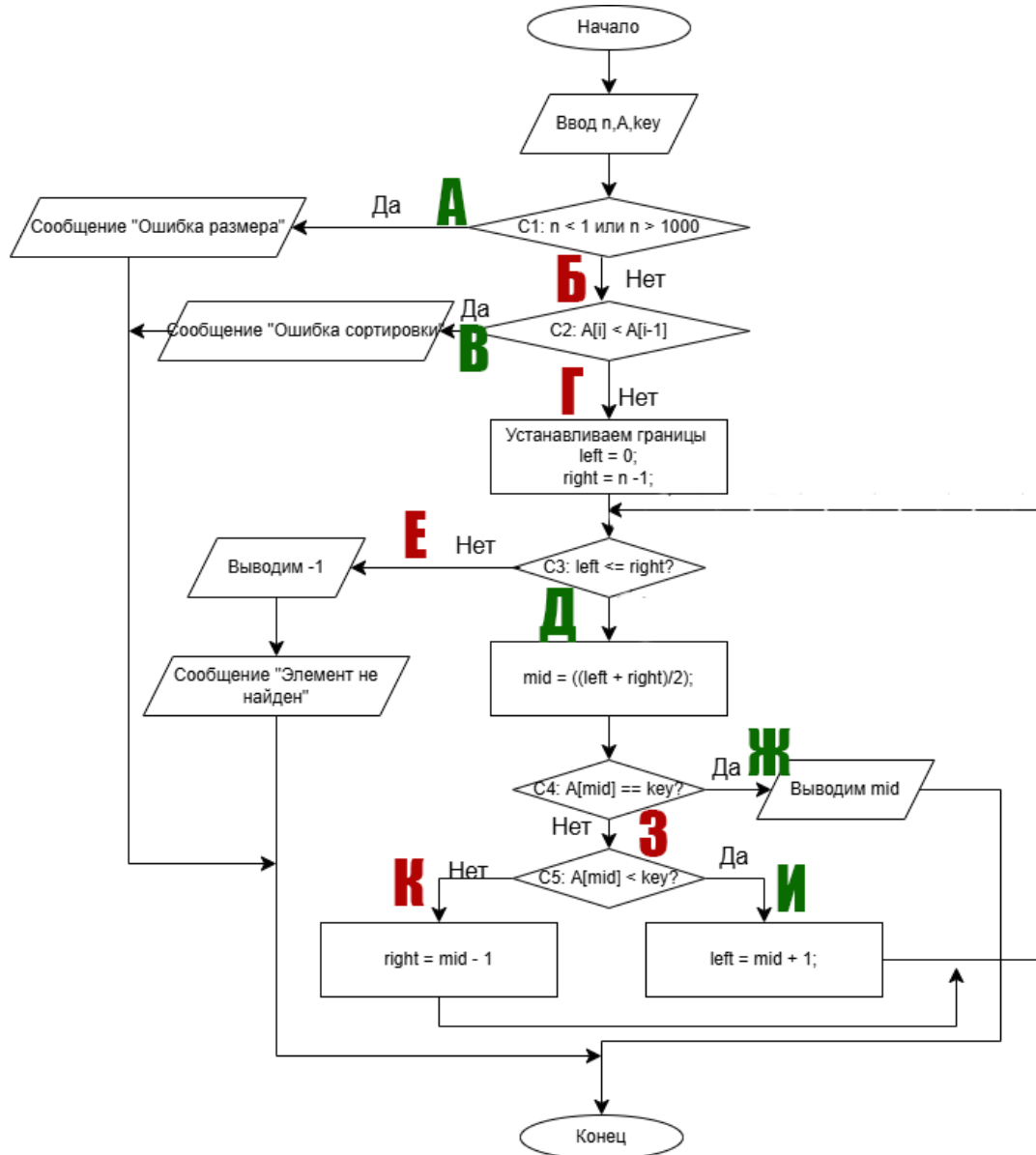


Рис. 3. Блок-схема

3.2 Тестирование методом «белого ящика»

Алгоритм составления тестов методом «белого» ящика предполагает обход всех возможных путей в теле программы и проверку выполнения каждого оператора не менее одного раза.

Для описания путей выполнения программы введём буквенные обозначения ключевых блоков и переходов на блок-схеме.

С1: Проверка условия ($n < 1 \parallel n > 1000$):

А – ветвь Т (ошибка размера -> вывод сообщения -> конец).

Б – ветвь F (продолжение).

С2: Проверка упорядоченности массива ($A[i] < A[i-1]$):

В – ветвь Т ($A[i] < A[i-1]$) -> ошибка сортировки -> конец.

Г – ветвь F (продолжение проверки / выход из цикла).

С3: Проверка условия $left \leq right$:

Е – ветвь F (выход из цикла -> элемент не найден -> вывод -1 -> конец).

Д – ветвь Т (вход в тело цикла).

С4: Проверка ($A[mid] == key$):

Ж – ветвь Т -> вывод индекса mid -> конец.

З – ветвь F.

С5: Проверка ($A[mid] < key$):

И – ветвь Т -> $left = mid + 1$ -> переход к проверке $left \leq right$.

К – ветвь F -> $right = mid - 1$ -> переход к проверке $left \leq right$.

3.2.1 Покрытие операторов

Критерием покрытия является выполнение каждого оператора программы хотя бы один раз. Это необходимое, но не достаточное условие для приемлемого тестирования по принципу белого ящика.

Для выполнения каждого оператора хотя бы один раз достаточно одного теста, который проходит по пути **Б** → **Г** → **Д** → **Ж** (успешный поиск).

Таблица 2

Покрытие операторов

N	n	A	key	Ожид. результат	Факт. результат	Путь	Статус
1	5	[1,2,3,4,5]	3	3	3	Б → Г → Д → Ж	Пройден

Тест №1 выполнил все операторы программы.

3.2.2 Покрытие решений

В соответствии с этим критерием необходимо составить такой набор тестов, при котором каждое условие в программе примет как истинное, так и ложное значения. Таким образом, к тестам, составленным для метода покрытия операторов, необходимо добавить тесты, которые будут проверять все возможные переходы.

Тесты, покрывающие все решения программы, представлены ниже.

Таблица 3

Покрытие решений

N	n	A	key	Ожид. результат	Факт. результат	Путь	Статус
1	5	[1,2,3,4,5]	3	3	3	$B \rightarrow \Gamma \rightarrow D \rightarrow \text{Ж}$	Пройден
2	5	[1,2,3,4,5]	6	-1	-1	$B \rightarrow \Gamma \rightarrow D \rightarrow \text{З} \rightarrow \text{И} \rightarrow D \rightarrow \dots \rightarrow E$	Пройден
3	0	—	—	Ошибка размера	Ошибка размера	A	Пройден
4	3	[3,1,2]	—	Ошибка сортировки	Ошибка сортировки	$B \rightarrow B$	Пройден
5	5	[1,2,3,4,5]	1	1	1	$B \rightarrow \Gamma \rightarrow D \rightarrow \text{З} \rightarrow K \rightarrow D \rightarrow \text{Ж}$	Пройден

Пояснение к путям:

- Тест №1 проверяет успешный поиск ($B \rightarrow \Gamma \rightarrow D \rightarrow \text{Ж}$).
- Тест №2 проверяет случай, когда элемент не найден: после нескольких итераций ($\text{З} \rightarrow \text{И}$) выход по E.
- Тест №3 (n вне диапазона) идёт по ветке A.
- Тест №4 (неотсортированный массив) идёт по ветке B.
- Тест №5 проверяет ветку K ($A[\text{mid}] > \text{key}$) – ключ меньше среднего.

Все решения (C1–C5) протестированы на T и F.

3.2.3 Покрытие условий

В соответствии с этим критерием количество тестов должно быть таким, чтобы все возможные результаты каждого элементарного условия в программе выполнялись по крайней мере один раз.

Для проверки элементарных условий внутри C1: $(n < 1)$ и $(n > 1000)$ используются тесты, представленные в таблице 4.

Таблица 4

Покрытие условий (C1)

N	n	A	key	Ожид. результат	Факт. результат	Путь	Статус
1	5	[1,2,3]	2	2	2	$B \rightarrow \Gamma \rightarrow D \rightarrow \text{Ж}$	Пройден
2	0	–	–	Ошибка размера	Ошибка размера	A	Пройден
3	1001	–	–	Ошибка размера	Ошибка размера	A	Пройден

Остальные элементарные условия проверяются следующими тестами:

Таблица 5

Покрытие остальных элементарных условий

Условие	Значение	Тест (n, A, key)	Путь	Статус
$(A[i] < A[i - 1])$	T	n=3, A=[3,1,2], key=5	$B \rightarrow B$	Пройден
	F	n=5, A=[1,2,3,4,5], key=3	$B \rightarrow \Gamma \rightarrow \dots$	Пройден
$(left \leq right)$	T	n=5, A=[1,2,3,4,5], key=3	D (вход в цикл)	Пройден
	F	n=5, A=[1,2,3,4,5], key=6	E (выход из цикла)	Пройден
$(A[mid] == key)$	T	n=5, A=[1,2,3,4,5], key=3	Ж	Пройден

Условие	Значение	Тест (n, A, key)	Путь	Статус
	F	n=5, A=[1,2,3,4,5], key=6	З	Пройден
$(A[mid] < key)$	T	n=5, A=[1,2,3,4,5], key=6	И	Пройден
	F	n=5, A=[1,2,3,4,5], key=1	К	Пройден

Все элементарные условия проверены на Т и F.

3.2.4 Покрытие решений и условий

Согласно этому критерию набор тестов является достаточно полным, если удовлетворяются следующие требования: каждое условие в решении принимает каждое возможное значение по крайней мере один раз, каждый возможный исход решения проверяется по крайней мере один раз и каждой точке входа управление передаётся по крайней мере один раз.

Тесты, написанные ранее, не покрывали ветвь **К** ($A[mid] > key$, $right = mid-1$). Поэтому добавлен тест №6.

Таблица 6

Покрытие решений и условий

N	n	A	key	Ожид. результат	Факт. результат	Путь	Статус
1	5	[1,2,3,4,5]	3	3	3	$B \rightarrow \Gamma \rightarrow D \rightarrow \text{Ж}$	Пройден
2	5	[1,2,3,4,5]	6	-1	-1	$B \rightarrow \Gamma \rightarrow D \rightarrow \text{З} \rightarrow \text{И} \rightarrow D \rightarrow \dots \rightarrow E$	Пройден
3	0	—	—	Ошибка размера	Ошибка размера	A	Пройден
4	1001	—	—	Ошибка размера	Ошибка размера	A	Пройден
5	3	[3,1,2]	—	Ошибка сортировки	Ошибка сортировки	$B \rightarrow B$	Пройден
6	5	[1,2,3,4,5]	1	1	1	$B \rightarrow \Gamma \rightarrow D \rightarrow \text{З} \rightarrow \text{К} \rightarrow D \rightarrow \text{Ж}$	Пройден

Все решения и условия покрыты (включая $C5 = T$ и $C5 = F$).

3.2.5 Комбинаторное покрытие условий

Этот критерий требует создания такого набора тестов, при котором каждая возможная комбинация результатов вычисления условий в каждом решении и каждая точка входа проверяются по крайней мере один раз.

Таблица 7

Комбинаторное покрытие условий

N	n	A	key	Ожид. результат	Факт. результат	Комбинация (n<1, n>1000)	Статус
1	5	[1,2,3]	2	2	2	(F, F)	Пройден
2	0	—	—	Ошибка размера	Ошибка размера	(T, F)	Пройден
3	1001	—	—	Ошибка размера	Ошибка размера	(F, T)	Пройден

Все возможные комбинации протестированы. Комбинация (T, T) невозможна для целого n.

3.2.6 Результаты тестирования методом «белого ящика»

В ходе тестирования программы бинарного поиска были разработаны тесты для различных критериев покрытия кода.

Для покрытия операторов был составлен один тест с входными данными $n=5$, массив $A = \{1,2,3,4,5\}$, $key=3$. Ожидаемый результат 3 совпал с фактическим, тест пройден. Этот тест выполнил все операторы программы, но не проверил ветви ошибок.

Для покрытия решений потребовалось пять тестов. Первый тест (путь $B \rightarrow \Gamma \rightarrow D \rightarrow \text{Ж}$) дал результат 3. Второй тест ($B \rightarrow \Gamma \rightarrow D \rightarrow 3 \rightarrow \text{И} \rightarrow \dots \rightarrow E$) с $key=6$ дал результат - 1. Третий тест (A) с $n=0$ проверил ошибку размера. Четвёртый тест ($B \rightarrow B$) с неотсортированным массивом $\{3,1,2\}$ проверил ошибку сортировки. Пятый тест

($B \rightarrow \Gamma \rightarrow D \rightarrow Z \rightarrow K \rightarrow D \rightarrow \text{Ж}$) с $\text{key}=1$ проверил ветку K ($A[\text{mid}] > \text{key}$). Все тесты пройдены.

Для покрытия условий протестированы элементарные условия в составе $C1$, а также условия $C2$ – $C5$ с помощью соответствующих тестов (см. подраздел «Покрытие условий»). Все тесты пройдены.

Для покрытия решений и условий использованы все вышеперечисленные тесты, включая тест с $n=1001$ для комбинации (F, T) и тест с $\text{key}=1$ для $C5=F$. Все шесть тестов пройдены.

Для комбинаторного покрытия условий проверены все возможные комбинации для $C1$. Все тесты пройдены.

4 Тестирование программы №2

4.1 Формальное описание программы

Автор: Юлия Гордиенко

Название: Проверка числа на палиндром

Дано: n – целое число.

Требуется: Определить, является ли число палиндромом (читается одинаково слева направо и справа налево и вывести результат). Отрицательные числа проверять без учёта знака минус.

Ограничения: n является целым числом в диапазоне $[-10^6; 10^6]$.

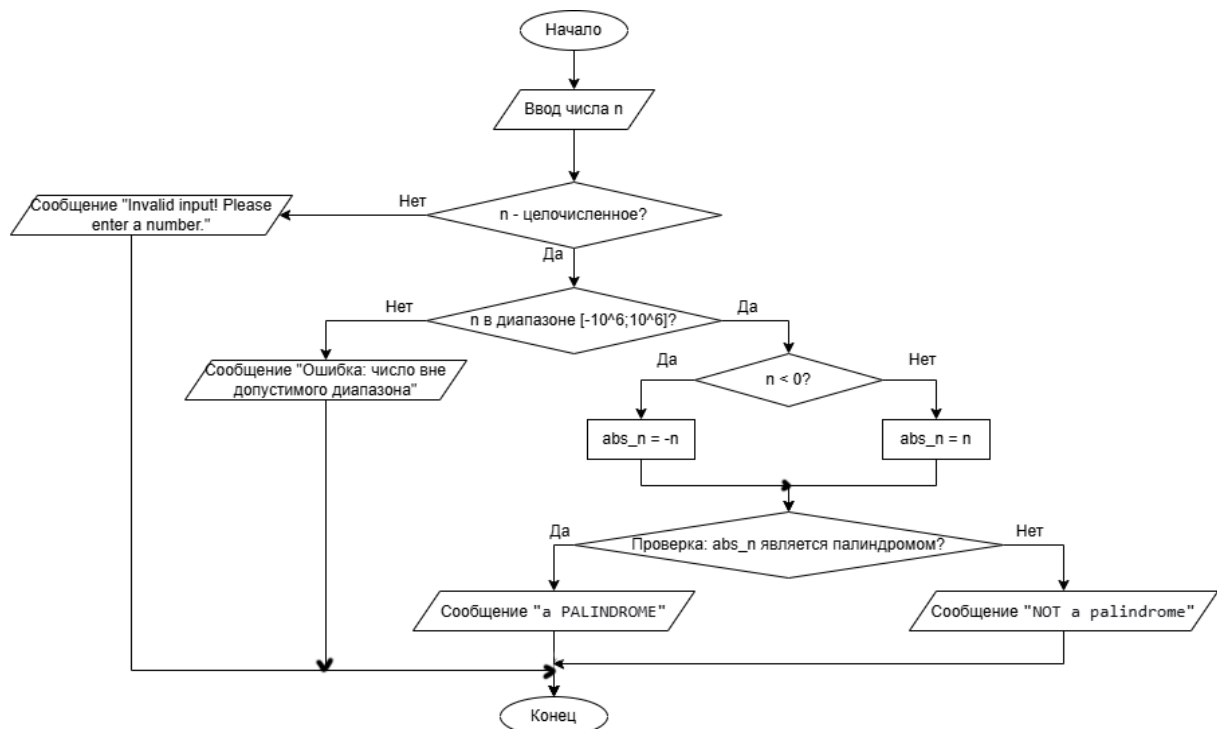
Спецификация:

Входные данные	Ожидаемый результат	Комментарий
0	a PALINDROME	Ноль является палиндромом
1, 5, 9	a PALINDROME	Однозначные числа — палиндромы
abc	"Invalid input! Please enter a number."	Ввод может быть только целочисленным
12.5	"Invalid input! Please enter a number."	Ввод может быть только целочисленным
11, 121, 1221, 12321, 1234321	a PALINDROME	Симметричные числа
-121, -1221, -12321	a PALINDROME	Проверка по модулю
10, 12, 123, 1234, 100, 12345	NOT a palindrome	Несимметричные числа
-10, -123, -1234	NOT a palindrome	Проверка по модулю
1000000	NOT a palindrome	1000000 -> 0000001 (1) != 1000000
-1000000	NOT a palindrome	Проверяется как 1000000 → не палиндром
1000001, 1000001	- Ошибка: число вне допустимого диапазона	Числа вне диапазона $[-10^6; 10^6]$

Примечание:

- 1) Программа не рассчитана на работу с числами, выходящими за пределы диапазона $[-10^6; 10^6]$.
- 2) Отрицательные числа проверяются по абсолютному значению (знак минус игнорируется).
- 3) Все однозначные числа являются палиндромами.
- 4) Числа, оканчивающиеся на 0 (кроме самого 0), не являются палиндромами.

Блок-схема алгоритма



4.2 Тестирование методом «чёрного ящика»

4.2.1 Разбиение на классы эквивалентности

Составлено разбиение на классы эквивалентности исходя из ограничений для программы.

Таблица 9

Классы эквивалентности для n

Входное условие	Допустимые классы	Недопустимые классы
Целое число n	Однозначное число-палиндром (1)	Нечисловое значение (6)
	Однозначное не-палиндром (не применимо, все однозначные — палиндромы)	Вещественное число (7)
	Положительный многозначный палиндром (2)	Целое число, выходящее за пределы диапазона $[-10^6; 10^6]$ (9)
	Положительный многозначный не-палиндром (3)	
	Отрицательный палиндром (4)	
	Отрицательный не-палиндром (5)	
	Ноль (0) (8)	

В таблице ниже приведены тесты, покрывающие все классы эквивалентности.

Таблица 10

Тесты, покрывающие классы эквивалентности

N	n	Кл. экв-ти	Ожидаемый рез.	Фактический рез.	Результат
1	7	(1)	а PALINDROME	а PALINDROME	Пройден
2	121	(2)	а PALINDROME	а PALINDROME	Пройден

N	n	Кл. экв-ти	Ожидаемый рез.	Фактический рез.	Результат
3	123	(3)	NOT a palindrome	NOT a palindrome	Пройден
4	-121	(4)	a PALINDROME	a PALINDROME	Пройден
5	-123	(5)	NOT a palindrome	NOT a palindrome	Пройден
6	abc	(6)	"Invalid input! Please enter a number."	"Invalid input! Please enter a number."	Пройден
7	12.5	(7)	"Invalid input! Please enter a number."	NOT a palindrome	Не прой- ден
8	0	(8)	a PALINDROME	a PALINDROME	Пройден
9	1000001	(9)	"Ошибка: чис- ло вне до- пу- сти- мо- го диа- па- зо- на"	"Ошибка: чис- ло вне до- пу- сти- мо- го диа- пазо- на"	Пройден

При вводе вещественного числа через точку (12.5) оператор `cin » userNumber` считает только целую часть до точки (12) и оставляет остаток во входном буфере. Это приводит к тому, что программа воспринимает 12 как допустимое число, проверяет его и выводит результат ("NOT a palindrome"), а затем из-за оставшихся в буфере символов (".5") снова срабатывает условие `!(cin » userNumber)` в начале сле-

дующей итерации цикла, что приводит к сбивчивому поведению и не соответствует ожидаемой реакции на ввод вещественного числа.

4.2.2 Анализ граничных значений

Для входных данных, представляющих собой целые числа, выделены следующие граничные значения:

- переход от однозначных к двузначным числам (9 и 10);
- переход от отрицательных однозначных к отрицательным двузначным (-9 и -10);
- переход через ноль (-1, 0, 1);
- границы, связанные с разрядностью (99 и 100, -99 и -100);
- границы диапазона (1000000, -1000000, 1000001, -1000001).

Для каждой границы определены тесты, представленные в таблице 11.

Таблица 11

Проверка граничных условий

N	n	Ожидаемый рез.	Фактический рез.	Результат
1	-1	NOT palindrome ^a	NOT palindrome ^a	Пройден
2	0	a PALINDROME	a PALINDROME	Пройден
3	1	a PALINDROME	a PALINDROME	Пройден
4	9	a PALINDROME	a PALINDROME	Пройден
5	10	NOT palindrome ^a	NOT palindrome ^a	Пройден
6	11	a PALINDROME	a PALINDROME	Пройден
7	-9	a PALINDROME	a PALINDROME	Пройден
8	-10	NOT palindrome ^a	NOT palindrome ^a	Пройден
9	-11	a PALINDROME	a PALINDROME	Пройден

N	n	Ожидаемый рез.	Фактический рез.	Результат
10	99	a PALINDROME	a PALINDROME	Пройден
11	100	NOT a palindrome	NOT a palindrome	Пройден
12	101	a PALINDROME	a PALINDROME	Пройден
13	-99	a PALINDROME	a PALINDROME	Пройден
14	-100	NOT a palindrome	NOT a palindrome	Пройден
15	-101	a PALINDROME	a PALINDROME	Пройден
16	1000000	NOT a palindrome	NOT a palindrome	Пройден
17	-1000000	NOT a palindrome	NOT a palindrome	Пройден
18	1000001	Ошибка: число вне допустимо- го диапазона	Ошибка: число вне допустимо- го диапазона	Пройден
19	-1000001	Ошибка: число вне допустимо- го диапазона	Ошибка: число вне допустимо- го диапазона	Пройден

Выводы: при тестировании граничных значений все тесты пройдены. Программа корректно обрабатывает переходы между разрядностями и знаками чисел.

4.2.3 Причинно-следственная диаграмма

Для построения причинно-следственной диаграммы введем следующие обозначения:

Таблица 12

Причины и следствия для причинно-следственной диаграммы

Причины	Следствия
C1 – ввод является целым числом	R1 – сообщение об ошибке ввода
C2 – ввод является вещественным числом (содержит точку)	R2 – сообщение об ошибке диапазона
C3 – ввод не является числом (строковые символы)	R3 – вывод “a PALINDROME”
C4 – целое положительное (или ноль) в допустимом диапазоне	R4 – вывод “NOT a palindrome”
C5 – целое отрицательное в допустимом диапазоне	
C6 – целое число вне диапазона $[-10^6; 10^6]$	
P – абсолютное значение числа является палиндромом	
Промежуточные узлы	
A = C4 OR C5	
B = C2 OR C3	

На основе выделенных причин и следствий построена причинно-следственная диаграмма (рис. 4). Диаграмма отражает логические связи между входными условиями и выходными результатами: например, следствие R3 (палиндром) достигается, когда истинны A (допустимое целое) и P (палиндром), а следствие R4 (не палиндром) – когда A истинно, но P ложно. Ошибки ввода (R1) возникают при $B = C2 \text{ OR } C3$, а ошибки диапазона (R2) – при C6.

Построенная диаграмма преобразована в таблицу решений (таблица 13), каждый столбец которой соответствует одному тесту. Такой подход позволил систематически охватить все комбинации причин и следствий, включая редкие сценарии.

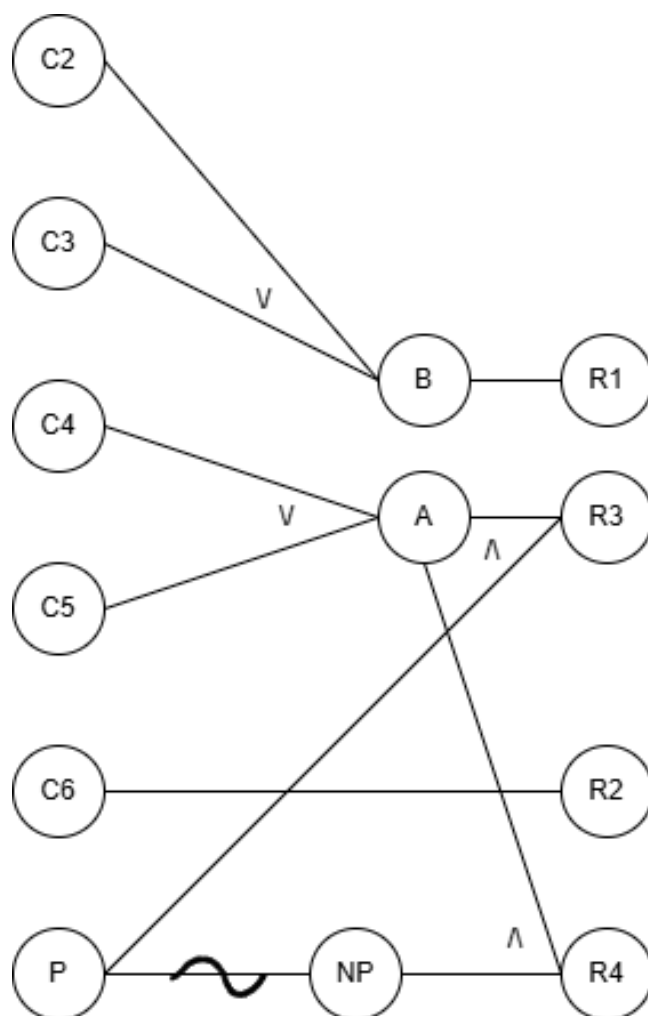


Рис. 4. Причинно-следственная диаграмма.

Таблица 13

Таблица решений

	1	2	3	4	5	6	7	8	9
C1 (целое)	1	1	1	1	1	1	1	0	0
C2 (вещественное)	0	0	0	0	0	0	0	1	0
C3 (не число)	0	0	0	0	0	0	0	0	1
C4 (положительное в диапазоне)	1	1	1	0	0	0	0	0	0
C5 (отрицательное в диапазоне)	0	0	0	1	1	0	0	0	0
C6 (вне диапазона)	0	0	0	0	0	1	1	0	0
P (палиндром)	1	0	1	1	0	—	—	—	—
A (C4 OR C5)	1	1	1	1	1	0	0	0	0

	1	2	3	4	5	6	7	8	9
B (C2 OR C3)	0	0	0	0	0	0	0	1	1
R1 (ошибка ввода)	0	0	0	0	0	0	0	1	1
R2 (ошибка диапазона)	0	0	0	0	0	1	1	0	0
R3 (палиндром)	1	0	1	1	0	0	0	0	0
R4 (не палиндром)	0	1	0	0	1	0	0	0	0

Вывод: таблица решений охватывает все возможные комбинации причин и следствий. Столбцы 1–3 соответствуют корректным положительным целым числам и нулю в диапазоне (палиндромы: многозначный, однозначный, ноль; не палиндром), столбцы 4–5 — отрицательным числам в диапазоне (палиндром и не палиндром), столбцы 6–7 — числам вне диапазона (положительное и отрицательное), столбцы 8–9 — ошибочному вводу (вещественное число и строка).

Таблица 14

Тесты на основе таблицы решений

N	Комбинация	n	Ожидаемый рез.	Фактич. рез.	Результат
1	Целое пол., диапазон, палиндром (многозначный) (C4=1, P=1)	121	a PALINDROME	a PALINDROME	Пройден
2	Целое пол., диапазон, не палиндром (C4=1, P=0)	123	NOT a palindrome	NOT a palindrome	Пройден
3	Целое пол., диапазон, палиндром (однозначный или ноль) (C4=1, P=1)	7	a PALINDROME	a PALINDROME	Пройден
4	Целое отр., диапазон, палиндром (C5=1, P=1)	-121	a PALINDROME	a PALINDROME	Пройден
5	Целое отр., диапазон, не палиндром (C5=1, P=0)	-123	NOT a palindrome	NOT a palindrome	Пройден
6	Целое пол., вне диапазона (C6=1)	1 000 001	Ошибка: число вне допустимого диапазона	Ошибка: число вне допустимого диапазона	Пройден

N	Комбинация	n	Ожидаемый рез.	Фактич. рез.	Результат
7	Целое отр., вне диапазона (C6=1)	-1 000 001	Ошибка: число вне допустимого диапазона	Ошибка: число вне допустимого диапазона	Пройден
8	Вещественное число (C2=1)	12.5	"Invalid input! Please enter a number."	NOT a palindrome	Не пройден
9	Не число (C3=1)	abc	"Invalid input! Please enter a number."	"Invalid input! Please enter a number."	Пройден

Вывод: при тестировании методом причинно-следственных диаграмм обнаружен непройденный тест №8 (ввод вещественного числа). Остальные тесты (1–7, 9) пройдены успешно.

4.2.4 Результаты тестирования методом «чёрного ящика»

В результате тестирования методом чёрного ящика было составлено **28 тестов** (9 классов эквивалентности + 19 граничных значений). Из них 1 тест не пройден (тест на ввод вещественного числа 12.5), остальные 27 тестов пройдены успешно.

Анализ обнаруженной ошибки: при вводе вещественного числа (например, 12.5) оператор `cin » userNumber` считывает только целую часть (12), обрабатывает её как допустимое целое число и выводит результат "NOT a palindrome". Оставшиеся во входном буфере символы (".5") вызывают ошибку ввода на следующей итерации цикла. Это поведение не соответствует спецификации, которая требует вывода сообщения об ошибке ввода.

Остальные тесты, включая все граничные значения и корректные классы эквивалентности, были успешно пройдены, что подтверждает корректность основной логики определения палиндрома.

5 Заключение

В ходе выполнения данной лабораторной работы были изучены методы тестирования «белым ящиком» и «чёрным ящиком».

В результате тестирования программы №1 (бинарный поиск) методом «белого ящика» все тесты были пройдены.

В результате тестирования программы №2 (определение палиндрома) методом «чёрного ящика» было составлено 28 тестов (9 классов эквивалентности + 9 граничных значений), из которых 27 были пройдены и 1 оказался непройденным. Непройдённый тест выявил ошибку валидации входных данных: при вводе вещественного числа программа ведёт себя некорректно из-за особенностей работы оператора ввода, что не соответствует спецификации. Это показало эффективность метода чёрного ящика для данной программы, позволив обнаружить несоответствие требованиям, которое не было бы найдено при тестировании белым ящиком.

По итогу выполненной работы можно сделать вывод, что оба метода тестирования могут быть полезны для выявления ошибок в программе, но каждый из них находит разные типы ошибок. Метод белого ящика проверяет корректность реализации внутренней логики, а метод чёрного ящика — соответствие поведения программы спецификации. Какой из методов окажется эффективнее для конкретной программы, заранее сказать невозможно, поэтому наиболее правильным решением будет проводить тестирование обоими методами в комплексе для обеспечения высокого качества программного продукта.

Выполняя эту работу, я убедилась, что тестирование — действительно необходимый инструмент поиска скрытых дефектов. Особенно наглядно это показал метод чёрного ящика: даже простая программа может давать сбой из-за неочевидных особенностей ввода.

Список использованной литературы

- [1] Майерс, Г. Искусство тестирования программ. – Санкт-Петербург: Диалектика, 2012 г.